

IIT Madras

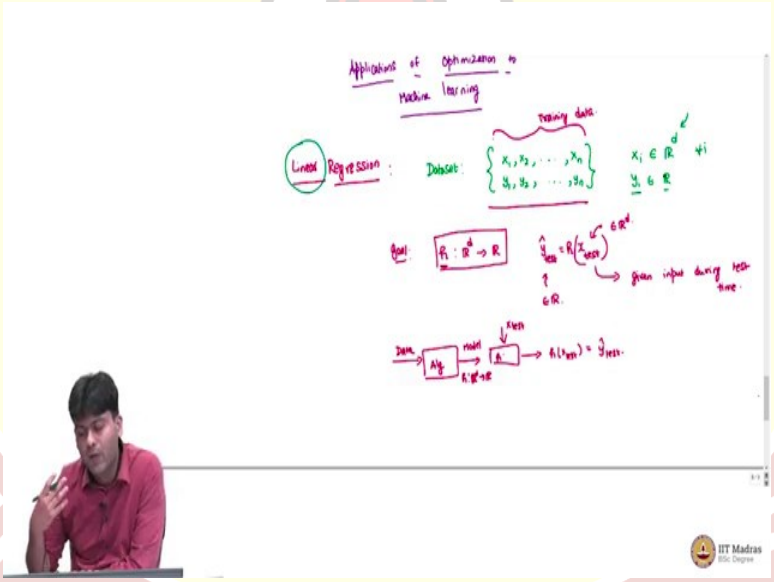
ONLINE DEGREE

Machine Learning Foundations
Professor Arun Rajkumar
Department of Computer Sciences & Engineering
Indian Institute of Technology, Madras
Applications of Optimization in Machine Learning

Okay, so what we have seen so far, are several properties of convex functions. Again, as I said these are not exhaustive properties, these are some properties which are relevant. And will prove useful, when you want to prove more complicated functions are convex, one of the property was sums of convex functions is convex, the second is about composition.

Now, what we are going to do next is try to look at a practical machine learning problem and see how whatever we have learned can actually be used to show easily, some of the functions that we will consider in machine learning are in fact convex functions. So, that is the goal going forward.

(Refer Slide Time: 1:04)



Applications of optimization in general to machine learning problems – So, of course when you look at a proper machine learning course, you will see lot of optimization problems, the goal of this course is to highlight one, or two important problems, to show that how whatever we have learnt actually will fit well into the optimization paradigm of machine learning and not really to illustrate all the important examples in machine learning.

So, the goal is to only highlight one, or two, complete application of optimization to machine learning. So, that is what we want to see now, and in this context, we are going to see a very

popular problem in machine learning, which is the problem of linear regression. So, I am sure you must have come across already this term in the earlier part of this course. So, what is the linear regression problem?

The linear regression problem is you are given a data set, which looks like this x 1, x 2, ., ., ., x n, and correspondingly y 1, y 2, ., ., ., y n, where your x'i is are your data points which have in general d features. So, you collect d features and you collect from let us say n different individuals, each person has d features. Let us say the height, weight, body mass index and so on. So, you collect a bunch of features and you collect d of them and that forms a data point, and you have n such data points.

Now, y i's are simply real numbers, this could be to predict how much can this person jump in a, let us say a high jump race for example. So, by looking at various features of this person maybe that is the problem, we are trying to solve, we want to predict that amount and this amount might take any potentially real value, well of course it will take any potentially positive value. But in general there might be other problems, where you might want to look at negative values as well. So, the general problem of linear regression is when y i is in general in R real numbers.

So, now how do we set up the problem. So, what is the goal? The goal is to find some function h. So, which maps R d to R, that is the goal of linear regression. And what do we do with this h? So, we will say for a new point x, so given x test a new point, then we will say the prediction y test, y hat test, where the hat indicates that you are actually predicting this, the algorithm predicts this would be simply h of x test.

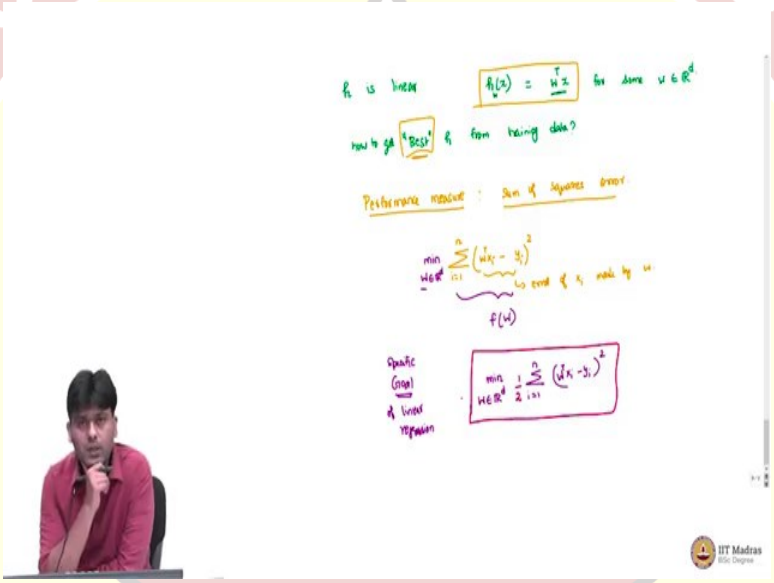
It is a given input during test time, this whole thing we will call as training data and during test time you are given a new test point and this belongs to R d. And if you have learnt a function h, from your training data, then you can use that function to make a prediction y hat test, which is also in real. This is a general goal of regression problems.

So, the general pipeline would be you have data and so this is some algorithm and out comes a model, which is simply a function h, from R d to R and then you get a x test as input, you apply the model h and output h of x test, which is y hat test. So, this is a general pipeline of a simple machine learning algorithm.

Now, the goal is how do you find this h, so from this data. So, you somehow want to you know summarize the data using a function. So, and then you do not want the find training data anymore potentially. So, you want to extract out the model from the data and then potentially throw away the data and only retain the function.

So, that you can, you can you know predict later on in the future, only using this function. The question is how do we learn this function and that is where this notion of linear regression comes into picture, where we are going to learn linear functions of x, that will predict y.

(Refer Slide Time: 6:23)



So, that is our goal, which means that in you are not learning in generally any function, but in linear regression our h is linear, which means what? So, your h of x, you can think of it as the simplest linear function, you can think of it as your h of x is w transpose x, for some w in R d, remember your x is in R d, you have d features and then you can think of these w, as giving some weights to these features, potentially positive, or negative weights, such that your prediction is simply a weighted sum of your features.

So, that is the that is the idea of linear regression. So, your h is actually characterized by some w. So, for every w you get a function h. So, if I fix w, then I know everything about this function, because that is the only thing that determines this function. And it not just determines it, determines in a linear fashion, that is why it is a linear function.

So, now how do we get this h best h. So, how to get the best h, how to get the best h, from training data? Well that is where we for into the optimization territory, we somehow have to fix what do we mean by best, for that we have to somehow fix what is the what do we mean by how good is a particular w with respect to the training data.

And one popular notion specifically the one that is used in least the regression problem is the performance measure called sum of squares. What does that mean? It simply means that, given a w. So, there are infinite w's and then we want to pick one w and argue that this is the best w for the data set, which means that for every w, we have to fix how good is this w with respect to the data set and that is what I call as performance measure of w.

And the famous, or common popular performance measure is the sum of squared error, which says that we look at the function, which is a sum of all data points, well the actual value you should be predicting for this data point is y i. But if you use a particular w, if I gave you a particular w, then you would actually be predicting w transpose x i, because that is the function functional form we have assumed.

So, this would be our prediction given by a particular w, on the data point x i, which is a real number, whereas the actual values y i. And now we somehow want to see how bad is this prediction with respect to the y i. And the simple way to see is to look at the squared error. And now this becomes a sum of i = 1 to n, w transpose $(x_i - y_i)^2$.

So, you can so now you can think of this as how good is w with respect to the data set. So, by way of measuring sum of squares of error, this is think of this as errors, error of x i made by a particular w. Now, of course once we have fixed how good is a w, then we can define what is the best w, well the best w is the one that has the least error.

So, you can simply say now that the best w would be the minimum over all w in R d, the one that minimizes this error. So, you can maybe define this, this itself as f of w, some function of w. And what we seek is a w, which minimizes $f(w)$.

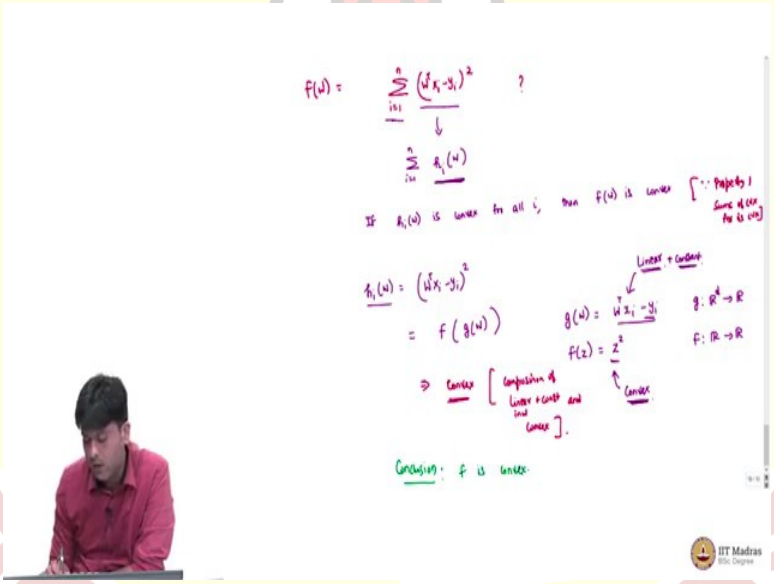
So, that is the goal, so then the more specific goal, so specific goal of regression of linear regression is to minimize over w in R d sum over i = 1 to n, w transpose $(x_i - y_i)^2$, for simply for convenience and we will see why in a minute sometimes we might want to assume minimize half of this, does

not really matter, because if some w minimizes the original function, the same w must necessarily minimize the some scaling of this as well.

So, you cannot have somebody else minimizing the scaled function. So, and so I can use any scaling and in this particular case, I am just using the scaling of half, that simply for some mathematical convenience, some terms will cancel out you can do it even without the half everything will work out, but let us see we use the half. So, this is our goal.

Now, the question is we are trying to minimize some function of w, it is a function from R to R d to R, where R because w is in R d and we have to think about what type of a function this is. So, we came up with this function by the intuitive notion, that it kind of minimizes the sum of squared errors, but what kind of a function is this.

(Refer Slide Time: 12:00)



So, in particular let us ask the question is this a convex function. so what is the convex function, that you are talking about. Let us say f of w is sum over i = 1 to n, w transpose x i - y i square, is this a convex function? This is the question we are asking.

Now, one way to do this to argue, or to see whether this is convex, or not is to you know write down the definition and then try to say that f of lambda w 1 + 1 - lambda w 2 is less than or equal to lambda f of w 1 + 1 - lambda f of w 2. But that might be pretty hard especially because this function does not look like simple I mean at least not so simple, it is a simple function, but not so simple.

But the question is can we, can we see if this is convex, or not using the properties of convex functions that we just discussed. We have discussed several properties of convex functions remember one of the property is that, the sum of convex functions is convex, which means that well this is the sum of multiple functions.

So, every data point you can think of it as giving a function, which is which measures for each w, how much error are you making on that particular data point. I can think of this itself as sum over i = 1 to n, h i of w. So, this h i simply measures how good is this w for this particular data points. now which means that if I can argue that the h i's are convex, then it is just a sum of convex functions, which we know is convex.

So, then one might want to argue one way to only want to focus on h i's. So, it is it suffices to focus on h i's, because if you can show that h i's, or convex then the sum is also convex, because we know sums of convex functions are convex. So, now if so let me write that down if h i of w is convex for all i, then f is convex, why, because sums of since property 1, basically. So, which is sum of convex functions is convex good.

So, now what is h i of w, h i of w is w transpose x i - y i square. Now, again you can try to plug in your standard definition of convex functions and then try to see if this is convex, or not perfectly reasonable way to do it. But we are going to not do that. So, we are going to observe that well what is this, what kind of a function is this?

So, this is simply a composition of sum f of g of w, where you can think of g of w, as w transpose x i - y i and f of w, as so f of x let me call it z, because should not confuse this x as z squared. So, where g is R d to R, f is R to R, f is just x, z squared.

Now, why is this composition, this function value. So, this is first given a w you evaluate w transpose x i - y i using g and then you square it using f. So, it is a composition so this is composition of two functions, where f is convex, this is clear we know that this is a convex function z squared is simple convex function.

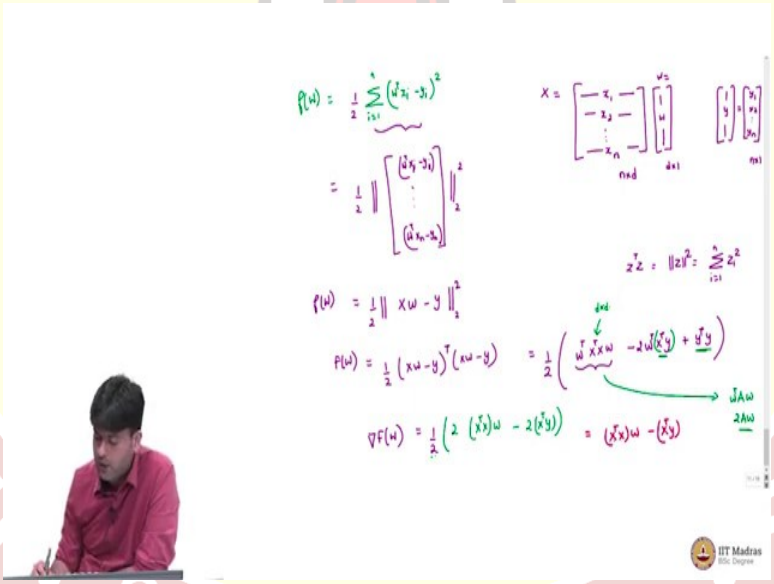
Now, what can we say about g? Well, it is simply w transpose x i and then translated using some value which is constant, which does not depend on w at all. So, now this is a linear function of w, why is this a linear function? Well, if I make w as 2 times w, then the I mean this is a linear + some constant which is also, this is linear + constant.

Now, that constant will not affect anything about the linearity. So, this is still a linear function. So, now what can I now use to argue that this whole thing is a convex function, well it is a composition of convex function and a linear + constant, which is linear in w. So, this is now I can use property 2, which I have used earlier to show that, this is which implies this is convex.

So, composition of a linear + constant, which is fine linear in w + constant and convex. So, we simply argued linear, but linear + constant does not really matter, you can go back and check that as well. So, this will also be a convex function.

So, now without really writing down the definition by simply noticing that, the function that we care about is a combination of a lot of smaller functions, which are easy to analyze, we have kind of argued that this the function, that we care about is actually convex. So, well the conclusion is that the sum of squares is a convex function. So, conclusion sum of squares f is convex.

(Refer Slide Time: 18:10)



Now, how does that help us? Well, it helps us now in two ways, the first way. So, now we can say that, if we can find a point, which is where the gradient is 0 of with respect to w, because as a function of w, then I know that, that is the point that minimizes this function. And so I would have gotten the optimal w and once I have the optimal w, I can use that w to predict on new data points as w transpose x.

So, which means that, now to get the w, we simply find the that point where the gradient is 0, we take the derivative and set it to 0. So, now let us look at the function again and see if we can do

this easily. The function is sum over i = 1 to n, w transpose x i - y i square, you can directly take the derivative of the gradient with respect to w, an easier and more convenient notation for this would be, now think of your x as capital X, as a matrix, which contains the data points.

So, as rows x 1, x 2, till x n and then you have a w, which is simply the w in R d and you have a y, let us call this y as simply y 1, y 2, till y n. So, your x is a so this is w, your x is a n cross d matrix, where each row is a data point, your w is d cross 1 vector, and your y is a n cross 1 vector. Now, we can write this whole thing as look at this vector w transpose x i - y, w transpose x 1 - y 1 square ., ., ., w transpose x n - y n square.

So, well let me just look at y n not even the squares, let me look at this, this vector. Now, this is simply capturing how much, what is the unsigned error. So, w transpose x i - y i, as a vector. So, this vector what dimension is this vector in? This is in R n, because for each data point I am looking at what is the prediction - what is the actual value and there are n data points. And now I look at the norm squared of this particular vector.

So, what is the norm of a vector squared in R n? So, that is simply going to be i equal to 1 to n, z i squared. So, which means that I can write this whole function as then as the norm of a vector squared. So, that is exactly this function. So, now how do I write this vector in this matrix notation? Well, I can write this vector itself as follows.

So, I can write this as this matrix x times w - the vector y norm square in L 2 norm square. So, remember x is the matrix with rows as data points. So, that is n cross d w is d cross 1 and so this is this adds up and we get exactly f of w. Now, I need to of course I had a half I was using a half, so I should always use that half even for these functions good.

Now, what is the norm of norm squared, you can also write the norm squared as a vectors transpose itself. So, that is another way to write the norm squared, which means I can write this whole thing as x w - y transpose x, w - y, they are exactly the same thing.

So, my f of w is in fact this function, which is z transpose z, where z is x w - y, which is equivalently. Now, if I expand this out this will look like w transpose x transpose x w - w transpose x transpose y, and then y transpose x w which is the same which will be twice + y transpose y, this would be my function. So, any function I given a w, I can I have just written the whole thing in matrix form.

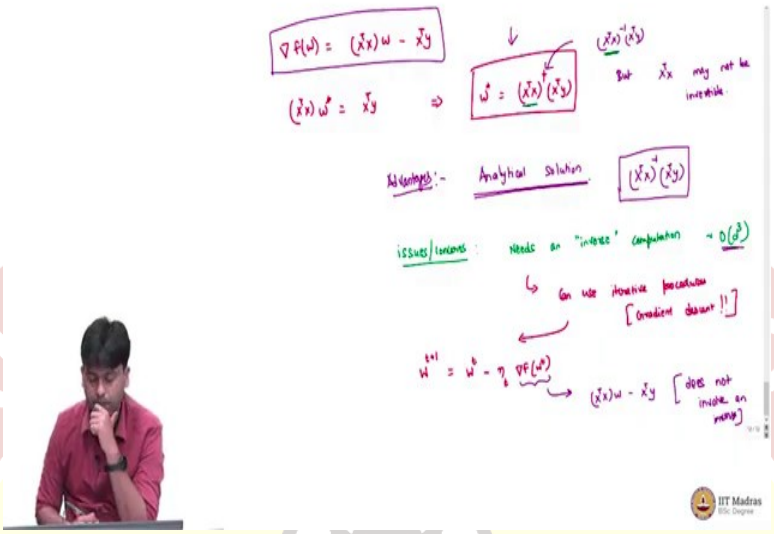
Now what I can do is? Now, I can ask what is the gradient of this function. So, I can look at the gradient, which is del f of w, which is what I want to compute. Now, this is going to be half times. Now, this whole function, the first, the first term is of the form w transpose sum matrix times w. Now, if I compute the gradient with respect to w and this is an exercise you should try out.

So, any a times w, so now ask what is the gradient of this function with respect to w, you can take the partial derivative with respect to each of the components. And it will turn out that that would simply be 2 times, this will be 2 times a into w, remember a into w is a vector in in R n sorry, R d. So, what is why is it R d x transpose x is x is n cross t, so x transpose x is d cross n times n cross t which is d cross t. So, this gradient is going to be a vector in R d. So, because it is a gradient with respect to w, w as an R d. So, the gradient will also be in R d.

So, this would be 2 times in our case x transpose sorry, 2 times x transpose x times w -, now this would be simply the gradient of this would be simply 2 times, this is some vector times . product with w, which is simply that vector itself, again that is another thing that you should try out. Because this is simply taking the partial derivative fixing every other term is constant.

So, you will simply get the ith term of x transpose y and so this is simply the vector of x transpose y. Now, y transpose y does not depend on w, so the gradient with respect to that is 0. So, this is going to be my gradient, of course now you see why I have used the half that is simply going to cancel out, these twos it is not really a big deal, just that it will look slightly neater. So, now this is the gradient of a function with respect to w. So, and it turns out to be x transpose x w - x transpose y good.

(Refer Slide Time: 25:08)



So, now let me rewrite that del f of w is x transpose x, w - x transpose y. Now, I want to find the w star, where this gradient is actually 0. So, because that would be because my f is convex, which we just showed, then this would actually be the optimal w star, the global optimum. Now, what would that be, well w star itself is going to be well x transpose x, w star x should equal x transpose y, because I am setting this to 0, and whichever w star that satisfy 0, w that satisfies 0 that let us call that w was w star.

So, this should satisfy which implies that w star itself is x transpose x. This is a pseudo inverse of x transpose y, well ideally, I should have had x transpose x inverse x transpose y, because I am multiplying pre multiplying I could multiply 3 multiply x transpose x inverse on both sides.

So, that the x transpose x cancels out, but the problem is x transpose x may not be invertible, invertible and so I am sure in the linear algebra part of the course, you should have seen that, if you do not have invertible matrix, you can still look at what is called as a pseudo inverse, which still satisfies the property that x transpose x pseudo inverse times x transpose x is i. So, it is the best possible guess for the inverse.

So, when an inverse really does not exist, why does the inverse does not exist? Because there is no one to one mapping. So, the x transpose x itself maps a lot of data a lot of vectors to the 0 vector, the null space is not just 0, it has a lot of other components.

So, you can potentially have a whole vector space of subspace of subspace of w, that satisfy x transpose x w star = x transpose y, and the pseudo inverse can be thought of as picking the vector with the least length in this subspace. So, at translated subspace, so that is the notion of pseudo inverse anyway.

So, this is a solution and the advantages of this is that, this is an analytical solution. So, we can compute the solution simply by setting the gradient to 0, you so this is just a function of the data. So, I mean let us simply assume at the moment that, this is an invert, this is an invertible matrix without loss of I mean well let us assume that invertible for the sake of discussion.

Otherwise, you can use the pseudo inverse. So, nothing would change in the discussion, but so this whole thing is a function of the data, I give you data points, which means that I am giving you the matrix x and also the vector y and from that you can simply compute this w.

Now, this is a completely analytical way to compute your w. So, and which means that, you do not really have to do any iterative way to find w star technically, because given data set i, this is just a function of the data set and it is a closed form expression, I can do this easily usually.

Now, the question comes then that well we saw all this whole machinery of gradient descent converging to the optimal value, when function is convex, but then at least for this particular case in machine learning problem, it looks like we do not really need to use the gradient descent algorithm at all, because we have a closed form solution, which is true and not true. Because this is yes we have a closed form solution.

But issues, or rather concerns with this particular closed form solution is that, it needs an inverse computation. So, you to do this, to solve this problem you have a pseudo inverse, or an inverse computation and usually the inverse computation of a matrix is not that computationally efficient. So, this is something like order of d cubed.

So, where d is the dimension of the d cross d is the matrix dimension. So, our x cross x transpose x is a d cross d matrix, d is the number of features and this inverse is going to typically take order of d cube, you can maybe make it slightly faster. But then usual best possible algorithms, that we know take more than d squared, may be slightly less than d cubed. But then nevertheless you can think of it as d cube.

And when d is very large, so when you have huge number of dimensions and this is something that will is very common in machine learning problems, for instance think of a face recognition type of a problem, where you take as input an image and this image you need to convert this image into pixels, and these pixels will be your features.

And if you have a high res image high resolution image and the number of pixels are going to be really large and each pixel has RGB values. So, the number of features themselves are going to be huge and even if, even if you have features in like let us say million features. Now, what this is saying is that, to compute this analytical solution I need to do like a million cubed kind of an operation and that might be computationally prohibitive for a lot of applications.

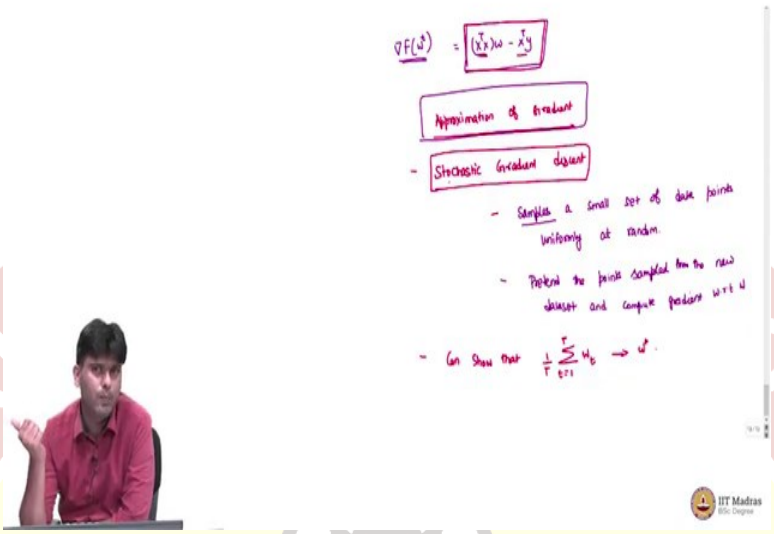
So, now the question is can we somehow get rid of this problem, where I know there is an analytical solution. But simply because of computational potential barrier, I will not be able to compute this, how else can I, how else can I get around this problem? Well now what comes as handy is, so can use iterative procedures, especially simple things like gradient descent.

So, gradient descent kind of comes to rescue why? Now, let us look at what the gradient descent would do? The gradient descent would update would be you have a running a running value for w. So, it is an iterative algorithm and w t + 1 is going to be your previous guess for w, which is w t - sum eta t, which is a step size times, the gradient of function at w t.

So, this is this would be my gradient descent update. And what is del f of w t, well at any w del f of w is simply x transpose x times w - x transpose y. So, this is simply an x transpose x times w - x transpose y operation and notice that this does not in involve an inverse, involve an inverse computation. And that is the biggest advantage of doing a gradient descent, because now at every step you can compute the gradient without explicitly computing an inverse at any point in the algorithm.

And we know that this algorithm will still converge to the same optimal solution, which is given by w star. And we have done this without computing the inverse at any point, well of course we did not compute it in one shot, we had to do it in an iterative fashion nevertheless in the iterative fashion in the iterative procedure, we never used an inverse computation. So, in general this can be much faster than doing an order of d cube kind of inversion. So, this is one point to keep in mind, the other point would be.

(Refer Slide Time: 33:53)



So, now the question is well is the gradient descent algorithm good enough, well now what could be a bottleneck in doing the gradient descent, well the only bottleneck could be in computation of the gradient itself, which we know is x transpose x w - x transpose y. And now the question is you have an x transpose y calculation, which has to be done. Now, if you have x transpose x times w times - x transpose x y.

Now, the question is can we even make this faster? Can we even not compute this whole thing at every step, can we make it even faster? Well, the solution is that, you can do that and the way to do that would be to not even compute this gradient explicitly, but then only compute an approximation of this gradient, approximation of gradient.

And the method to do that, is using a technique called as stochastic gradient descent. And what you do here is that, you at every round at what is what would your gradient descent do, it would start with the w t and compute the gradient at w t and then move in that direction. And to do this w t you need to do x transpose x times w - x transpose y.

Now, the stochastic gradient descent algorithm would say that, this x transpose x is kind of using my entire data set to compute x transpose x. Now, your n the number of data points you might have could be let us say in billions. So, even x transpose x computation might be really hard. So, because it has to be it is an n cross d times d cross n matrix, even computing that might be prohibitively large.

So, what does stochastic gradient decision do, it at every step all it does is, it samples a small set of data points, uniformly at random, at every round in your gradient descent algorithm, you sample a small set of data points from the billion data points. And now pretend as if this is my data set. So, you forget all the rest of the points, you pretend that this is my data set and compute the gradient of the function w with respect to only this data set.

So, pretend the point sampled are the new form the new data set and compute gradient with respect to w. So, I am throwing away a lot of points, which means that my gradient that I am computing is not with respect to the entire data set, but with only respect to a small set of items. But then these data points are sampled uniformly at random.

So, you can think of the gradient as in fact an approximation of the actual gradient, that you should have computed using the entire data set. But now because the data set is too large computing the gradient is expensive. So, I am going to not compute the entire gradient with respect to the all the data points, but then only with respect to a small set of data points, which is more manageable with the computational power I have.

Now, the great thing is that, now every round you are getting an approximate gradient and remember every round it is not the same set of small data points, that we are using. So, every round we are using a different set of points, because we are sampling.

So, you are uniformly sampling. So, somehow you are not moving in the gradient direction, but your move actual gradient direction, but you might move in a different direction. But then at every round you will kind of move slightly away. Or from the gradient, but then the errors might eventually cancel out and that is the hope. And that is the intuition for why this such an algorithm might actually work.

So, we would not really prove this in this course, but can show that you know the average, let us say you run this algorithm for capital T rounds and you look at w t, where t = 1 to t, where you get this w's by computing this approximate gradients, approximate gradients at every step, we can show that this average is actually will go to w step. So, this the average will get better and better.

So, this is a very powerful algorithm, then because now you have avoided the inverse by saying that I am going to go from analytical computation to a gradient based solution. And you are also avoiding computing the entire gradient, but then I am going to make only a small set of sample

only a small set of points, which means that I also gain with respect to the computational power at every step.

So, and the nice thing is that, you can also show good guarantees about this, we want to really go into the entire guarantees of this algorithm. But then we can show that, the in some sense the average data point w t's will actually be get better and better.

Now, so this will give us a very, very practical algorithm for solving huge machine learning problems and as you can see that the stochastic gradient descent algorithm is motivated through simple optimization ideas, that we started looking at including iterative methods and convex functions properties, that iterative methods will lead to good solutions.

So, far to summarize we have looked at convexity as a general idea, and then we are try to apply it to certain machine learning problems, where we see if you set up your optimization problem as a convex problem, then iterative solutions exist and even you can practice you can even you make them better using some kind of randomness.

So, that is the high level take away from this discussion, one thing we did not see and which we will see next is, so though we have looked at convexity and when we have optimality conditions and so on, we have no not looked at problems, where there are constraints. So, all the convex problems that we have looked at so far did not have any constraints on the parameter, for instance the linear regression problem did not have any constraints on the w, that we are looking for.

But there are machine learning problems, where you might want to constraint your parameter space to some nice set, where you want to look for the best possible solution and which means that we want to understand, what are the optimality conditions for you know convex functions under constraint setting, which is what we will discuss in the later classes. But I think this kind of concludes the non-constrained unconstrained convex problems and how they can be applied to real practical machine learning algorithms. Thank you.